



P12

**Evidence-Grading-Driven
Reproducible Agent Systems
Engineering: A Longitudinal
Empirical Study of an
AI-Co-developed Collaboration
Engine Across 30+ Versions**

证据分级驱动的可复现 Agent 系统工程

Evidence-Grading-Driven Reproducible Agent Systems Engineering: A Longitudinal Empirical Study of an AI-Co-developed Collaboration Engine Across 30+ Versions

This paper is a meta-study/empirical report of the UDOS Reasoning Engine v7.5.0 (main tag v7.5.0, commit 1a71270). The material is this repo's complete development archive: git history, CHANGELOG (38 version entries), docs7/VERIFICATION.md, reports7/*.json, 236 tests, past packaging records. The three-level evidence labels verified / cpu-proto / unverified run through code, reports, documentation layers. This paper contains real negative results and one real version-number incident, all honestly preserved. Exemplar contrast positioning: this paper is the Stage B upgrade draft of the "UDOS Writing and Layout Specification v2." The reproduction appendices (Appendices A/B/G) actively echo the Kaplan exemplar "appendices centrally place summary tables + supplementary experiments" and the NSA exemplar "reproduction configs fully written" practices: concentrating all key numbers, hardware fingerprints, seeds, gate assertions into lookup tables, the concrete numbers of external exemplars (Kaplan/NSA/AlexNet) being for this paper unverified methodological references.

Abstract (Structured, Four Parts)

[Background & Problem] In AI-co-developed research software, "number drift"—quietly upgrading a prototype number to "measured" in documentation, inflating multipliers, passing a prototype off as production—is a primary enemy of reproducibility. AI tends to generate fluent, "plausible" narrative and does not naturally append "CPU prototype, single seed, unmeasured" qualifiers.

[Method] Taking a research engine co-developed with AI across 30+ minor versions as a single case, this paper reports a mandatory evidence-grading engineering method: every quantitative claim carries a verified / cpu-proto / unverified label consistently across code (EvidenceGrade), reports (evidence_grade field), and documentation, backed by a "no benchmark no claim" discipline, a single-source-of-truth version, and baseline-JSON regression gates.

[Evidence & Results] Using longitudinal archive evidence, we report the method's shape, cost, and failure points: (1) negative results are systematically kept—identifiability skill for "acceleration a" is -0.508 (estimator worse than the mean baseline), naive Sim2Real success is only 0.5, and CPU-bound throughput peaks at 2 agents then drops ($1.59\times$ peak); (2) the discipline itself has failure points—from v7.4.3 to v7.4.10, eight consecutive versions stayed at v7.4.2 because the version string carried a "v" prefix the replacement regex missed, only fixed in v7.5.0 when a test exposed it; (3) under extreme faults the system honestly reports failure rather than faking success.

[Contribution] A real, cross-30+-version longitudinal engineering archive containing counter-examples; honest preservation of negative results and the method's own failure points; evidence grading treated as a first-class engineering output that flows with data, is consistent across three layers, and is guarded by tests. This is a single-case empirical report, not a universal causal claim.

Keywords: reproducibility; evidence grading; AI-co-developed software engineering; empirical report; negative results; version engineering

1 Introduction

1.1 Problem: Number Drift in AI-Co-development

AI co-development is changing how research software is produced: developers describe requirements in natural language, AI rapidly generating code, documentation, even "seemingly reasonable" performance numbers. While this productivity greatly improves, it also brings a new reproducibility risk—**number drift**: a number measured at the prototype stage quietly upgraded to "measured" in later documentation; a $\times 2$ prototype speedup written as "hundredfold improvement"; a CPU deterministic simulation narrated as "production-grade deployment." Claims in external reporting that "some Agent platform already has some capability" (all unverified reporting caliber) further amplify this risk. [CITATION NEEDED: reproducibility crisis machine learning reporting evidence grading]

This paper cares not about "whether AI can write code" but "how to keep numbers honest in AI co-development." We believe this needs an **institutional** method, not reliance on developer consciousness: attaching an evidence grade to every quantitative claim, consistent across code, reports, documentation, and guarded by tests in CI.

1.2 Research Questions

- **RQ1:** In long-cycle iteration, which types of "overclaim" actually occurred, how were they intercepted by gates?
- **RQ2:** Can the three-level evidence labels (verified / cpu-proto / unverified) consistently marked across code, reports, documentation suppress number drift?
- **RQ3:** What are the costs and failure points of this evidence discipline itself?

Making these three research questions more concrete: RQ1 pursues the concrete form of "overclaim"—numbers inflated? prototypes passed off as production? negative results deleted? This repo's archive lets us answer version by version. RQ2 pursues whether three-layer consistency really works—it is not just "we wrote labels" but "whether labels are consistent across code-reports-documentation and guarded in CI." RQ3 pursues cost and failure—evidence grading is not free, it slows development and makes expression conservative; and it itself can fail (e.g. the version-number incident). Raising RQ3 separately is to avoid this paper becoming "an advertisement for evidence grading"—we must honestly tell its cost and its failures.

1.3 Contributions (conclusion map)

- **A real, cross-30+-version, counter-example-containing longitudinal engineering archive:** this paper is not methodological deduction but longitudinal coding of a real engine's development archive from v0.1 to v7.5.0.
- **Keeping negative results:** honestly reporting negative results like acceleration skill -0.508 , naive Sim2Real 0.5, CPU throughput $1.59\times$ peaking then dropping.
- **Honestly reporting the method's own failure points:** the v-prefix version-number incident (8 consecutive versions stuck at v7.4.2) is this paper's most important self-exposure—it proves "convention" does not equal "guarding."

1.4 Why Talk About This Now

AI co-development maturity is moving from "whether code can be generated" to "whether a research system can be maintained long-term." A research engine often iterates dozens of versions, spanning

multiple (including AI) collaborators. In this long cycle, what most easily happens quietly is not code bugs but **number semantic drift**: the same " $1.59\times$ speedup," at the prototype stage being CPU-bound 2-Agent peak, in marketing docs possibly becoming "any Agent count can $1.59\times$ speed up"; the same "million-scale matrix," at the closed-form extrapolation stage being "protocol complexity upper bound," in docs possibly becoming "million Agents measured running." What this paper wants to establish is the engineering institution where this drift is intercepted by grade labels at every claim.

1.5 Scope Statement

This paper is a single-case empirical report, the author being the developer, with confirmation bias. We do not claim "evidence grading works in all projects," only reporting its form, cost and failure points in this one project. The audience is researchers doing AI co-development, reproducible software engineering, and research systems engineering.

2 Related Work

2.1 Reproducibility Crisis and Evidence Grading

Machine learning and scientific computing have much discussion of the reproducibility crisis: model cards, datasheets, claim-evidence tracking and other methods try to make numbers more honest. [CITATION NEEDED: model cards datasheets reproducibility ML reporting] Homologous with the main line of "how to honestly measure LLM capability," the evaluation community developed a progressive evidence-grading spectrum: **conceptually**, a single score cannot characterize a model; **mechanistically**, Hendrycks et al.'s MMLU uses multiple choice across 57 subjects to measure knowledge breadth (Hendrycks, Burns, Basart et al., 2021, ICLR 2021, arXiv:2009.03300); Liang et al.'s HELM uses 30 models/42 scenarios/7 metrics for holistic evaluation, opposing "picking a single highest score to report" (Liang, Bommasani, Lee et al., 2022, arXiv:2211.09110); Chiang et al.'s Chatbot Arena uses crowdsourced Elo battles to measure human preference (Chiang, Zheng, Sheng et al., 2024, arXiv:2403.04132); White et al.'s LiveBench uses contamination-free dynamic benchmarks to measure real level, top models <70% (White, Dooley, Godinez et al., 2024, ICLR 2025, arXiv:2406.19314). **In evidence**, this spectrum shows: honest measurement needs multi-dimensional, controlled, contamination-resistant—which is precisely the methodological embryo of "numbers cannot be reported only picking good-looking ones." This paper continues this tradition but focuses on how evidence grading lands across code-reports-documentation in the new situation of "AI co-development."

Model cards and datasheets mainly solve "how model/dataset meta-information is recorded," facing a single artifact; while this paper faces a research engine iterating 30+ versions, numbers scattered across code, reports, documentation, CHANGELOG. This paper's three-level evidence grading can be seen as the transformation of the above evaluation spectrum (multi-dimensional, controlled, contamination-resistant) and model-card ideas in the "long-cycle, multi-version, AI co-development" scenario: not writing one card for a single artifact, but attaching a grade to every claim, letting this grade be consistent across three layers and guarded in CI.

2.2 Empirical Study of AI-Co-developed Software Engineering

Empirical studies of AI-assisted programming (e.g. LLM-generated code) focus on code quality, defects, productivity, but less on "how numbers in documentation drift over time." [CITATION NEEDED: LLM assisted software engineering empirical study code quality] Homologous with "automatic measurement also distorts" is LLM-as-a-Judge: Zheng et al. proved model judges' agreement with human judges

>80%, but with position bias, verbosity bias, self-preference (Zheng, Chiang, Sheng et al., 2023, NeurIPS 2023 Datasets, arXiv:2306.05685). **In evidence**, this research shows "automatic review" itself has systematic bias and must be identified and marked—which agrees with this paper's "evidence grades must flow with claims": any automatically produced number is not naturally trustworthy. This paper provides a longitudinal case, filling the "claim drift" dimension.

In particular, AI co-development has a risk traditional development lacks: in AI-generated documentation, performance numbers "seemingly reasonable but without benchmark support" may appear. Because AI tends to generate fluent, complete, "plausible" narrative, it does not, like a careful human researcher, naturally append "this is a CPU prototype, single seed, unmeasured" after each number. This needs engineering institutions to supply this "carefulness"—evidence grading is precisely this institution. One implicit argument of this paper is: AI co-development amplifies number-drift risk, therefore also amplifies the necessity of evidence grading.

2.3 Existing Gap

We hypothesize (to be confirmed/refuted by search): few studies take "three-level evidence labels + version single source + baseline regression gates + keeping negative results + self-exposing method failure points" as a complete engineering institution, and verify its form, cost and failure with cross-version real archives. [CITATION NEEDED: claim-evidence tracking versioning reproducibility empirical] This paper is a single-case empirical report in this direction.

2.4 Capability Evolution Main Line and Evaluation Benchmarks: Benchmarks Measure Capability, Evidence Grading Measures the Trustworthiness of Claims Themselves

To understand this paper's position, one must first see the LLM/Agent capability stack's evolution main line, then see how evaluation evolves. **Mechanistically**, this main line starts from representation and backbone: Attention Is All You Need replaces recurrent structure with self-attention (Vaswani, Shazeer, Parmar et al., 2017, NeurIPS 2017, arXiv:1706.03762), BERT uses bidirectional pretraining to establish the representation paradigm (Devlin, Chang, Lee, Toutanova, 2019, NAACL 2019, arXiv:1810.04805); then GPT-3 uses 175B scale and few-shot in-context learning to show emergent capability (Brown, Mann, Ryder et al., 2020, NeurIPS 2020, arXiv:2005.14165); **post-training/alignment**, InstructGPT uses SFT→reward model→PPO three-stage RLHF so a small aligned model outperforms an unaligned large one (Ouyang, Wu, Jiang et al., 2022, NeurIPS 2022, arXiv:2203.02155), DPO further uses closed-form preference optimization to remove the explicit reward model (Rafailov, Sharma, Mitchell et al., 2023, NeurIPS 2023, arXiv:2305.18290); **reasoning**, Chain-of-Thought uses intermediate reasoning steps to elicit multi-step reasoning (Wei, Wang, Schuurmans et al., 2022, NeurIPS 2022, arXiv:2201.11903).

In evidence, synchronized with capability evolution, evaluation benchmarks also evolve: MMLU uses multiple choice across 57 subjects to measure knowledge breadth (Hendrycks, Burns, Basart et al., 2021, ICLR 2021, arXiv:2009.03300); Codex/HumanEval uses 164 handwritten programming problems needing independent run verification to measure function correctness, Codex solving 28.8%, 100 samples 70.2% (Chen, Tworek, Jun et al., 2021, arXiv:2107.03374); HELM uses 30 models/42 scenarios/7 metrics for holistic evaluation (Liang, Bommasani, Lee et al., 2022, arXiv:2211.09110). **The difference from UDOS** is precisely here: benchmarks (MMLU/HumanEval/HELM) measure "how strong the model's **capability** is"; while this paper's evidence grading measures "how trustworthy the **claims** the system writes themselves are." The two are two layers: the former is the measured object's output quality, the latter the narrative quality about the measured object. A model may have a high MMLU score, but its project documentation may still write a CPU prototype as "production-grade"—precisely the "number

drift" this paper governs. In other words, benchmarks answer "whether it works," evidence grading answers "you say it works, is that claim trustworthy."

Organizing this causal chain from "data in" to "capability evaluated" into Table 3 (thread from the Reference Master Library "12 Mechanism Panorama Concept Thread Table," all master-library  verified entries):

Table 3 LLM/Agent Capability Stack Mechanism Thread and UDOS Landing

#	Mechanism link	Core problem solved	Core classic (arXiv)	UDOS landing
1	Token/Embedding	open vocabulary, text to vector	BERT(1810.04805)	input representation layer
2	Attention	long-range dependency, parallel modeling	Attention(1706.03762)	sequence backbone
3	Transformer/FFN	deep stacking, gating capacity	ResNet/GLU	network skeleton
4	MoE/sparse	large params small compute	Switch(2101.03961)	expert legion/ routing
5	pretraining/scale	emergence and scaling laws	GPT-3(2005.14165)、 Chinchilla(2203.15556)	base and compute budget
6	post-training/ efficient fine-tuning	alignment cost, multi-task adaptation	InstructGPT(2203.02155)、 LoRA(2106.09685)	plugin/alignment
7	alignment/ preference	acting per human preference	InstructGPT(RLHF)、 DPO(2305.18290)	lightweight alignment/circuit breaker
8	reasoning	multi-step, verifiable thinking	CoT(2201.11903)、R1(2501.12948)	reasoning chain/self-play
9	RAG/external knowledge	facts external, traceable update	RAG(2005.11401)	external knowledge handoff
10	tool calling/action	connecting external API and environment	Toolformer(2302.04761)、 ReAct(2210.03629)	Agent action
11	multi-agent/ consensus	collaboration, Byzantine resistance, self-organization	AutoGen(2308.08155)、Contract Net、PBFT	P1/P2/P10
12	evaluation/ evidence grading	objective measurement, contamination prevention, reproducibility	MMLU、HELM、Chatbot Arena、 LiveBench、HumanEval	P12 evidence grading

Table 3 note: the 12 mechanism-thread rows are all Reference Master Library  verified entries (verified 2026-09-19); row 12 is this paper's landing—"measuring claim trustworthiness itself" distinguished from benchmarks "measuring model capability."

This table's last row (link 12) is precisely this paper's position: after the capability stack evolves from Token all the way to multi-agent action, "how to honestly measure, prevent contamination, be reproducible" itself becomes a link needing governance. This paper does not repeat benchmarks' work but takes "measuring claim trustworthiness itself" as the research object—this is the division of labor between evidence-grading methodology and capability evaluation benchmarks.

3 Method: Evidence-Grading Engineering Institution

3.1 Three-Level Evidence Labels

This repo divides every quantitative claim's evidence grade into three levels:

- **verified**: this repo's fixed-seed rerunnable measurement, numbers from reports7/*.json and pytest.
- **cpu-proto**: CPU deterministic prototype/simulation, Agents not real LLMs, no real network/GPU.
- **unverified**: external reporting/vendor caliber, only motivation and contrast, not cross-evidenced with this repo's numbers.

These three levels are consistent across code (`EvidenceGrade` enum), reports (`evidence_grade` field), documentation (every number attached a grade). Three-layer consistency can be written as a requirement:

```
$$ \forall \text{claim } x: \text{grade}(\text{code}(x)) = \text{grade}(\text{report}(x)) = \text{grade}(\text{doc}(x)). \tag{1} $$
```

3.1.1 Meaning of Three-Layer Consistency

Three-layer consistency is not as simple as "code writes verified, documentation also writes verified." It means: the same number, the grade in code comments, the `evidence_grade` field in reports7 JSON, the marking in docs7 documentation paragraphs, must point to the same grade. If JSON is cpu-proto but documentation writes "measured," even if the number itself is correct, it is one "claim upgrade." This repo's practice is letting JSON be the single source, documentation reading numbers from JSON, grades also following JSON, mechanistically avoiding "documentation writing cpu-proto as verified."

3.1.2 Boundary Between verified and cpu-proto

This paper repeatedly emphasizes a boundary: verified means "fixed seed, rerunnable, test guarded"; cpu-proto means "CPU deterministic prototype, Agents not real LLMs, no real network/GPU." The difference is not "accuracy" but "whether it can represent real deployment." A cpu-proto million-scale matrix extrapolation is correct in protocol complexity, but it does not mean "a million real LLM Agents are already running." Evidence grading's role is letting the reader, on reading this number, immediately know what it represents and does not.

3.2 "No Benchmark No Claim" Discipline

Any performance conclusion must have corresponding reports7 JSON and tests; numbers without benchmarks cannot be written into conclusions. The baseline-JSON regression gate prevents performance numbers silently regressing: each release, key metrics compared against the previous baseline, regression failing the gate.

3.3 Version Single Source

The version number is single-source defined in `udos7/__init__.py`, `pyproject.toml` etc., avoiding multi-place inconsistency. But as the Section 6 incident shows, the single-source "convention" needs test guarding.

3.4 Form of Evidence Labels in Code

In code, evidence labels are not just comments. It is an enum (`EvidenceGrade`), function return values directly carrying the `evidence_grade` field. This means: when report scripts get results from functions, the grade flows with the data, not handwritten afterward. In the dictionary `run_mission` returns,

"evidence_grade": "cpu-proto" and "accepted": 24 are parallel fields; report scripts write the whole dictionary into JSON, the grade naturally entering JSON. This "grade flows with data" design is harder to tamper than "separately marking grades in documentation"—because the grade is part of the data, not part of the narrative.

3.5 Form of Evidence Labels in Reports

In reports7 JSON, every key report's top level carries `evidence_grade`. For example `matrix_scale_demo.json` top level is "cpu-proto," `v7_verification.json` is "verified." Readers opening JSON see at first glance what grade this number is, not needing to find it in documentation. This "report carries its own grade" practice makes the data itself self-describing.

3.6 Form of Evidence Labels in Documentation

In docs7 documentation and this paper, every key number has its grade and limitation noted after it. For example "million-scale fan-in 9, rounds 14 (cpu-proto, analytical extrapolation, not measured)," readers immediately know this is closed-form extrapolation rather than real running. Documentation does not "launder" grades—it preserves JSON's grade as-is and writes out limitations.

3.7 Why Three-Layer Consistency Matters

Three-layer consistency's key is: it makes "upgrading a claim" no longer a point action. If someone wanting to write cpu-proto as verified needs to simultaneously change code return values, JSON fields, documentation narrative in three places; and these three places are produced by different flows (code, report scripts, writing) and have CI tests aligning. This structure raises the cost of "quiet upgrading," thereby suppressing number drift.

4 Data and Methodology

4.1 Data Sources

- `CHANGELOG.md`: 38 version entries (`## v` count).
- `reports7/v7_verification.json`: v7.0.3 world-model verification, 967,796 params, blind MSE 0.01037.
- `reports7/agent_scaling.json`: CPU-bound throughput curve (2-Agent peak 244.17 t/s, speedup 1.594; 4-Agent 1.034; 8-Agent 0.695).
- `reports7/splat_transfer_demo.json`: naive Sim2Real 0.5, robust 1.0, margin 0.22.
- Full regression 236 items all green.

4.2 Longitudinal Coding

Extract all quantitative claims by version, coding as {grade, source, rerunnable, later corrected}. This paper is a descriptive single case, not reporting p values.

4.1.1 Coding Unit

This paper's coding unit is "one quantitative claim," not "one version." For example "2-Agent peak 244 t/s, 1.59×" is one claim; "blind MSE 0.01037" another. Each claim is pursued with four questions: what is its grade in code? what is its source field in reports7 JSON? can it fixed-seed rerun? was it corrected or removed in later versions? This coding makes "claim drift" observable rather than impressionistic.

4.1.2 Coder and Bias

This paper is a single coder (author being developer), with the expert bias of "I know how this number came then"—I may be more tolerant of this repo's numbers. The hedge: all coding points to concrete files and fields (see Appendix B evidence ledger), readers can check themselves; for doubtful claims, this paper tends to downgrade rather than upgrade.

5 Results

Per the v2 specification, the results chapter unfolds by "experimental setup → contrast (negative results vs robust/control) → sensitivity (gates and regression) → validity threats (Chapter 8)."

5.0 Version Milestones and Evidence Discipline Evolution

This repo's CHANGELOG contains 38 version entries, from early v0.x prototypes to v7.5.0's matrix closed loop. Table 2 excerpts some key milestones and their evidence grades, showing how evidence discipline evolves with versions.

Table 2 Key Version Milestones and Evidence Grades (excerpt; evidence grades marked row by row)

Version	Module/capability	Key number	Evidence grade
v7.0.3	world-model verification	967,796 params, blind MSE 0.01037	verified
v7.2.2	Agent legion throughput	2-Agent peak 244 t/s (1.59×)	cpu-proto
v7.3.9	Sim2Real	naive 0.5, robust 1.0, margin 0.22	cpu-proto
v7.4.6	layered tree	million-scale fan-in 9, rounds 14	cpu-proto
v7.4.8	circuit-breaker trio	three-level circuit breaker, rollback, reassignment	cpu-proto
v7.5.0	matrix closed loop	24/24 closing, 3 faults isolated, 236 tests	cpu-proto

Table 2 note: the closer to bottom mechanisms (world-model verification) the easier to get verified; the closer to "large-scale/real deployment" (million matrix, Sim2Real) the more cpu-proto. Evidence discipline's role is precisely preventing the latter being upgraded to verified in documentation.

This table shows: the closer to bottom mechanisms (world-model verification), the easier to get verified; the closer to "large-scale/real deployment" (million matrix, Sim2Real), the more cpu-proto. Evidence discipline's role is precisely preventing the latter being upgraded to verified in documentation.

5.1 Negative Results Kept

Table 1 Key Negative Results Kept (evidence grades marked row by row)

Metric	Value	Meaning	Source
acceleration a identifiability skill	−0.508	estimator worse than mean baseline	v7_verification.json
naive Sim2Real success rate	0.5	half fail without margin	splat_transfer_demo.json
CPU-bound throughput (8 Agents)	0.695×	dropping after peak	agent_scaling.json

Table 1 note: the three negative results are all numbers "defining system capability boundaries," source JSON and evidence grades marked row by row; keeping them rather than deleting is for boundaries to be seen.

Read Figures 1, 2, 3.

Below, deeply analyze these three negative results one by one, explaining why keeping them is more valuable than "deleting."

Acceleration a skill is -0.508 . In v7.0.3's identifiability analysis, we estimate several hidden dynamics parameters and quantify with "skill = estimator error improvement relative to mean baseline error." The "acceleration a" parameter's skill is -0.508 (its $\text{mae}=0.9256$, $\text{mae_mean_predictor}=0.6138$, $n=160$), meaning: using the estimator to guess this parameter is worse than directly using the mean baseline. This is a clear negative result—it tells us, under current observation conditions, the acceleration parameter is simply not identifiable (or the non-identifiable direction is opposite our hypothesis). If we deleted this number and only reported "spring ω skill 0.426, other v2 skill 0.82," readers would think all parameters identifiable. Keeping -0.508 makes the boundary of "which parameters identifiable, which not" clear. This is precisely the typical value of "keeping negative results" in evidence discipline.

Naive Sim2Real success rate 0.5. In v7.3.9's Sim2Real experiment, the naive (no margin) strategy success rate is only 0.5, average collision 0.5; while after adding robust margin 0.22, success rises to 1.0, collision falls to 0. If we only reported the robust version, readers would think Sim2Real naturally feasible; keeping naive's 0.5 makes the "necessity of margin" conclusion have a contrast. This negative result also shows: Sim2Real's difficulty is not "whether it can transfer from simulation to real" but "whether to keep a safety margin when transferring."

CPU-bound throughput $1.59\times$ peaking then dropping. In v7.2.2's Agent legion throughput benchmark, CPU-bound tasks peak at 2 Agents 244.17 t/s (speedup 1.594), drop at 4 Agents to 1.034, further at 8 Agents to 0.695. This is a typical "performance turning point": CPU-bound tasks constrained by physical cores, GIL, torch threads, concurrency beyond parallelism not rising but falling. If we only reported $1.59\times$, readers would think "more Agents faster"; keeping 0.695's drop makes the "CPU-bound not bigger-is-better" boundary clear. This negative result also distinguishes CPU-bound and I/O-bound: the latter (using `asyncio.sleep` to simulate remote LLM waiting) indeed near-linearly scales to $7.986\times$, but that is simulation, not real GPU/API.

These three negative results' common point is: they all define system capability boundaries. Deleting them makes documentation "prettier" but more misleading. Evidence discipline's value lies in letting these boundaries be seen.

CPU-bound throughput hump

Figure 1 CPU-bound throughput hump. Horizontal axis concurrent Agent count (1/2/4/8), vertical axis speedup relative to single Agent; solid line CPU-bound local inference (constrained by this machine's 2 threads), peak $1.594\times$ @ 2 Agents, then 4/8 Agents dropping to 1.034/0.695. Evidence grade cpu-`proto` (single-seed pilot). Source `reports7/agent_scaling.json`.

Parameter identifiability skill

Figure 2 Parameter identifiability skill contrast. Horizontal axis each hidden parameter (v0/acceleration a/spring ω /other v2), vertical axis `identifiability_skill` (>0 better than mean baseline, <0 worse); acceleration a is -0.508 (the only negative, i.e. "not identifiable" negative result), spring ω 0.426, other v2 0.82, v0 0.318. Evidence grade verified (`v7_verification.json`, fixed seed, 21 gates all passed).

Sim2Real naive vs robust

Figure 3 Sim2Real naive vs robust contrast. Left naive (no margin) success 0.5, average collision 0.5; right robust (margin 0.22) success 1.0, average collision 0.0. Evidence grade cpu-`proto`

(`splat_transfer_demo.json`, real world using fixed-seed perturbation synthetic proxy, not real machine/MuJoCo).

5.2 Version-Number Incident (the Method's Own Failure Point)

The CHANGELOG v7.5.0 entry records: previously `udos7/__init__.py` and `pyproject.toml` stayed at v7.4.2, because the version replacement regex expected no "v" prefix while the actual version number carried "v," causing version replacement from v7.4.3 to not take effect, 8 consecutive versions (v7.4.3 to v7.4.10) externally still showing v7.4.2, until v7.5.0 being exposed by a test then fixed. This incident proves: **"version single source" is only a convention, not guarding; conventions must be enforced by tests, otherwise they silently fail.**

This incident deserves expanded analysis because it is a textbook case of "evidence discipline itself failing." On the surface, this repo already did "version single source": version numbers concentrated in `udos7/__init__.py` and `pyproject.toml`, other places reading from there. But the problem arose at the "auto-update version number on release" step: the regex used to replace version numbers was written in the "no v prefix" format, while the actual version number was written as a v-prefixed string ("v7.4.3"). The regex did not match, replacement silently failed, so from v7.4.3 to v7.4.10 these eight versions, the code's externally reported version number was always v7.4.2. This error had no error—regex replacement failure throws no exception, it just did nothing. Until v7.5.0 added a "test asserting version number equals expected" test, this silent error was exposed.

This incident has three lessons for evidence-grading methodology: first, **conventions cannot replace tests**. "We agree version single source" is a promise, but promises do not error when the regex is wrong; only "test asserting version number equals expected" does. Second, **silent failure is more dangerous than erroring**. In this incident, the system did not crash, tests did not go red (the version assertion test not yet added), documentation kept updating, only the version number quietly wrong for eight versions. Third, **evidence grading itself needs grading**. If we ourselves cannot guarantee the accuracy of the most basic metadata like "version number," then our honesty claims about other numbers also need a "we too make mistakes" discount—precisely why this paper publishes this incident.

5.3 Honest Failure (kill-switch)

The matrix end-to-end under "all-domain faults + zero retry budget" honestly triggers the kill-switch, reports not closed (accepted<24, governance.ok=false), not faking success. This is evidence discipline's runtime manifestation.

This "honest failure" deserves separate discussion because it is isomorphic with evidence grading's role in documentation: evidence grading in documentation prevents "prototype passing off as production," the runtime kill-switch prevents "system passing off as success." When all-domain specialists are broken and retry budget zero, the system's most "beautiful" approach is to force a closed result, but this system chooses to stop, report not closed, leave a complete audit. This "staying honest also at runtime" design is evidence discipline extending from "documentation layer" to "runtime layer." It shows: evidence grading is not just a writing norm, it should permeate the system's failure-mode design.

5.4 How Three-Layer Consistency Lands in Practice

Taking v7.5.0's matrix numbers as an example, three-layer consistency lands like this: in code the dictionary `run_mission` returns directly carries `evidence_grade: "cpu-proto"`; `scripts7/matrix_demo.py` writes this dictionary into `reports7/matrix_scale_demo.json`, JSON top level also carrying `evidence_grade: cpu-proto`; `docs7/TOPLOGY_MATRIX_v7.5.0.md` and this paper read numbers from JSON and note in text "evidence cpu-

proto, single-seed pilot." The three layers point to the same grade, no single layer can alone upgrade the number. If someone wants to change `cpu-proto` to "measured" in documentation, they need to simultaneously change code return values, JSON fields and documentation—and all three have test guarding in CI, changing one mismatching another.

5.5 Baseline-JSON Regression Gate

This repo's another mechanism is baseline-JSON regression. Each key version's verification metrics stored in reports7 JSON (e.g. `v7_verification.json`'s 967,796 params, blind MSE 0.01037). Next release running `verify_v7.py`, if key metrics silently regress (e.g. blind MSE much worse than previous), the gate fails. This prevents the classic problem of "changing along, performance quietly degrades, nobody noticing." As of v7.5.0, full regression 236 items all green—itsself evidence of the gate working.

5.6 Test Count Evolution

From v7.4.1 to v7.5.0, the topology layer added 90 tests (`test_v741` to `test_v750`, per-file count $8+8+9+9+7+9+10+11+10+9=90$). These tests are not just "functional tests," they are also "evidence guard tests": `explicit_matches_formula` guards explicit-closed-form mutual verification, `TraceLedger.verify` guards hash-chain integrity, version assertion tests guard version numbers. Test count growth is essentially "evidence trustworthiness" growth—each test added, one more number or behavior fixed and guarded.

5.7 Claim Drift Curve: A Longitudinal Coding

This paper did a longitudinal coding of the CHANGELOG's 38 version entries: every quantitative claim marked as {grade, source, rerunnable, later corrected}. One overall observation is: **the closer to v7, the more conservative claims**. Early v0.x documentation had numbers like "1000×," "50,000 neurons" without benchmark support; at v7 refactoring, these numbers were removed, replaced by reports7 JSON carrying `evidence_grade` fields, rerunnable, test guarded. This "claim drift curve"—from exaggeration to conservatism—is precisely evidence discipline's trajectory with versions.

Of course, this observation itself is single-case, single-coder, with subjective bias. A stricter "claim drift curve" needs external contrast projects and double coding, future work ([RESULT NEEDED: use=claim drift curve quantitative, double-coding κ]).

5.8 Gate Ledger: What Is Not Achieved, Honestly Recorded

This repo maintains a "resource gate" ledger, clearly recording which capabilities not yet achieved: GPU/HPC (about ¥35,400/month tier) not achieved, so 0.5B+ end-to-end, MuJoCo-MJX, complete 3DGS, vLLM KV benchmark all outside this report; multi-vendor LLM API keys not provided, so real multi-model cross-scoring not done; real collection devices not available, so real egocentric data closed loop not done. This gate ledger itself is part of evidence discipline: it writes clearly "what we cannot do," rather than writing "what we want to do" as "what we did."

6 Discussion and Threats

6.1 Cost

Evidence discipline has costs: slower development, conservative expression, every number needing a grade. But it reduces rework—numbers like "1000×," "50,000 neurons" in early documentation removed in v7 refactoring, precisely evidence discipline intercepting overclaims.

6.1 Concrete Forms of Cost

Evidence discipline's cost is not abstract, embodied in three places. First, **conservative expression**: a number can only write what it actually represents. For example "million-scale matrix" can only be written as "closed-form extrapolated protocol complexity upper bound," not "million Agents measured running." This conservatism makes documentation read "not exciting enough," but it prevents misleading. Second, **every number needs a grade**: when writing each performance claim, going back to confirm its `evidence_grade` in reports7 JSON and marking in documentation. This adds writing friction. Third, **test and gate maintenance**: baseline-JSON regression gates need maintaining, each release comparing key metrics, regression failing the gate; version-number assertion tests need writing. These are all real engineering costs.

6.2 Benefits: What Was Intercepted

Whether the cost is worth it depends on what it intercepted. This repo's evidence discipline intercepted at least three types of overclaims: first, **prototype numbers upgraded to measured**: numbers like "1000×," "50,000 neurons" without benchmark support in early documentation removed in v7 refactoring—precisely the direct result of "no benchmark no claim" discipline. Second, **CPU simulation passing off as GPU capability**: this repo clearly distinguishes CPU-bound and I/O-bound, clearly marking complete 3DGS, MuJoCo-MJX, real-machine Sim2Real needing GPU/HPC gates, not passing off these capabilities in CPU reports. Third, **external number cross-evidencing**: all external industry numbers in this paper (GPT-6 Astra, π 0.5, Dyna-2 etc.) marked unverified, reporting caliber, not cross-evidenced with this repo's numbers.

6.3 Cost-Benefit Tradeoff

We believe, for a research engine—its core output being "trustworthy numbers"—evidence discipline's cost is worth it. If a research engine's numbers are untrustworthy, its code's elegance loses meaning. Evidence discipline's essence is treating "number trustworthiness" as a first-class output, not as post-writing decoration. This agrees with the "research systems engineering" positioning: a research engine is not a product, its value lying in "whether others can believe your numbers."

7 Conclusion

This paper, with cross-30+-version real engineering archives, reports an evidence-grading-driven reproducible Agent systems engineering method: three-level labels running through code-reports-documentation, keeping negative results, using tests to guard version single source, and honestly reporting failure at runtime. The most important lesson is: evidence discipline is not a set of conventions written in README but an institution needing tests, gates, baseline regression to jointly guard; even so, it can fail (e.g. the v-prefix incident), failure points must be honestly recorded. This paper hopes to provide an honest sample for reproducible engineering in the AI co-development era.

7.1 Three Actionable Points

For engineering teams doing AI co-development, this paper provides three actionable points: first, let evidence grades flow with data (function return values carrying grade fields), not handwritten afterward in documentation; second, write "no benchmark no claim" and baseline regression gates into CI, letting silent regression be caught by tests; third, keep negative results and the method's own failure points as

first-class content, not delete as "ugly things." These three points all come from this repo's verified practice, not empty talk.

7.2 Honest Boundary

This paper must re-emphasize: this is a single-case, author-as-developer empirical report. We do not claim evidence grading works in all projects, do not claim it can eliminate all number drift, do not claim the v-prefix incident is the only failure point. We only honestly report: in this one project, this method looks like this, cost how much, failed where. Readers should judge for themselves whether it applies to their projects.

8 Limitations and Validity Threats (separate chapter)

T1 Single-case threat (most primary). This paper is single-case, author as developer, with confirmation bias. **Most likely counterexample:** this method cannot be executed in a project with greater commercial pressure and a larger team. **How this paper guards:** publishing all archives, keeping negative results and incidents; without external contrast only submitting to empirical report tracks, not making universal causal claims.

T2 No external contrast. This paper did not contrast this project with other similar open-source Agent projects, so cannot answer "whether evidence grading really makes projects more honest than those without it." This contrast needs finding 3–5 similar projects, coding their documentation claims' evidence-grade completeness, double coding and reporting κ ([RESULT NEEDED: use=double-coding consistency κ]).

T3 Self-report bias. The biggest threat is "self-report bias"—we may selectively tell good stories. Hedges: first, publishing all archives (git history, CHANGELOG, reports7, tests), letting readers check themselves; second, keeping negative results and incidents (−0.508, 0.5, 1.59× peak, v-prefix incident, kill-switch failure), not only telling success; third, clearly marking which are single-seed pilots, which to supplement.

T4 External validity. We cannot say "evidence grading works in all AI co-development projects." It works in this project, partly possibly because this project is small, the author familiar with archives, and the research engine positioning naturally values number trustworthiness. In a customer-facing commercial project, market pressure may make "delete negative results" temptation greater, evidence discipline execution cost possibly higher.

T5 Coding bias. Single coder, with the expert bias of "I know how this number came then." Hedge: all coding points to concrete files and fields; for doubtful claims tending to downgrade rather than upgrade.

References

A. Verified Literature (verified through the Reference Master Library 2026-09-19, fields copied from master library)

1. Hendrycks, D., Burns, C., Basart, S. et al. (2021). Measuring Massive Multitask Language Understanding (MMLU). ICLR 2021. arXiv:2009.03300. (knowledge breadth benchmark across 57 subjects)

2. **Liang, P., Bommasani, R., Lee, T. et al.** (2022). Holistic Evaluation of Language Models (HELM). arXiv:2211.09110. (holistic evaluation methodology of 30 models/42 scenarios/7 metrics)
3. **Chiang, W.-L., Zheng, L., Sheng, Y. et al.** (2024). Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. arXiv:2403.04132. (crowdsourced Elo battles/human preference)
4. **White, C., Dooley, S., Godinez, S. et al.** (2024). LiveBench: A Challenging, Contamination-Free LLM Benchmark. ICLR 2025. arXiv:2406.19314. (contamination-free dynamic benchmark)
5. **Zheng, L., Chiang, W.-L., Sheng, Y. et al.** (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685. (automatic review and its position/verbosity/self-preference biases)

Capability evolution main line (post-training/alignment/reasoning, Section 2.4):

1. **Vaswani, A., Shazeer, N., Parmar, N. et al.** (2017). Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762. (self-attention replacing recurrent structure)
2. **Devlin, J., Chang, M.-L., Lee, K., Toutanova, K.** (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019. arXiv:1810.04805. (bidirectional pretraining representation paradigm)
3. **Brown, T. B., Mann, B., Ryder, N. et al.** (2020). Language Models are Few-Shot Learners (GPT-3). NeurIPS 2020. arXiv:2005.14165. (175B scale and few-shot emergence)
4. **Ouyang, L., Wu, J., Jiang, X. et al.** (2022). Training Language Models to Follow Instructions with Human Feedback (InstructGPT). NeurIPS 2022. arXiv:2203.02155. (RLHF three-stage alignment)
5. **Rafailov, R., Sharma, A., Mitchell, E. et al.** (2023). Direct Preference Optimization (DPO). NeurIPS 2023. arXiv:2305.18290. (closed-form preference optimization removing reward model)
6. **Wei, J., Wang, X., Schuurmans, D. et al.** (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903. (multi-step reasoning elicitation)

Capability evaluation benchmarks and tools/actions (Section 2.4 and Table 3):

1. **Chen, M., Tworek, J., Jun, H. et al.** (2021). Evaluating Large Language Models Trained on Code (Codex/HumanEval). arXiv:2107.03374. (HumanEval function correctness benchmark; Codex solving 28.8%, 100 samples 70.2%)
2. **Lewis, P., Perez, E., Piktus, A. et al.** (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (RAG). NeurIPS 2020. arXiv:2005.11401. (facts external, traceable update)
3. **Schick, T., Dwivedi-Yu, J., Dessì, R. et al.** (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023. arXiv:2302.04761. (self-supervised learning to call APIs/tools)
4. **Yao, S., Zhao, J., Yu, D. et al.** (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629. (reasoning traces and actions interleaved)

B. To-Verify Literature (not collected or verified in master library, keeping placeholders, not to be completed from memory)

1. [CITATION NEEDED: reproducibility crisis machine learning reporting evidence grading] — candidate: reproducibility crisis and evidence grading survey.
2. [CITATION NEEDED: model cards datasheets reproducibility ML reporting] — candidate: model card/datasheet methods.
3. [CITATION NEEDED: LLM assisted software engineering empirical study code quality] — candidate: LLM-assisted software engineering empirical study.

4. [CITATION NEEDED: claim-evidence tracking versioning reproducibility empirical] — candidate: claim-evidence tracking and version reproducibility empirical.

Appendix A Reproduction Commands (echoing NSA "reproduction configs fully written")

```
cd release-v5.4.4/udos-engine && git checkout v7.5.0 # 1a71270
pytest tests7/ -q # 236 items all green
python scripts7/verify_v7.py # recompute v7_verification.json
```

Reproduction config snapshot (hardware/software/seeds, per NSA exemplar "base scale/layers/dimensions/hyperparams/hardware/optimizer/seeds" fully written): Python 3.12.11, PyTorch 2.14.0+cpu, 2 threads, Linux CPU, device=cpu; world-model ckpt worldmodel_v7.0.3.pt (window=6, hidden=256, n_layers=2, scene_dim=32, use_kinematics=true); dataset train 1280 windows/256 trajectories, val/test/calib each 640 windows/128 trajectories; fixed seed, coverage tolerance ± 0.05 (fan ± 0.10), 21 gates all passed.

Appendix B Evidence Ledger (spot-check 5 traceable numbers; echoing Kaplan appendix parameter lookup)

#	Number	Source
1	acceleration skill -0.508	v7_verification.json estimator_param_error.accel_a
2	naive Sim2Real 0.5	splat_transfer_demo.json sim_to_real.naive.success_rate
3	CPU peak $1.594 \times (244.17 \text{ t/s})$	agent_scaling.json headline
4	967,796 params	v7_verification.json params
5	38 version entries	CHANGELOG.md ## v count

Appendix C Author's Intended Statements

- **Target journals/conferences:** candidates ICSE SEIP/MSR, FSE, EMSE, JSS, ACM Computing Surveys. Quartiles/IF/deadlines [to check] .
- **Data and code availability:** Apache-2.0; tag v7.5.0 (commit 1a71270); CHANGELOG, reports7, tests all public.
- **AI use statement:** draft AI-assisted, author responsible for checking numbers and scientific meaning; external numbers all marked unverified.

Appendix D Internal Review Record (five-dimension reviewer self-assessment)

1. **Novelty (8/10):** real, long-term, counter-example-containing AI co-development engineering archives scarce.
2. **Rigor (7/10):** numbers traceable; deduction for single case, no external contrast.
3. **Evidence strength (7/10):** negative results and incidents honestly kept a highlight.

4. **Relevance (8/10)**: reproducibility is the core topic of systems research in the AI era.
5. **Writing (7/10)**: structure complete, self-exposing failure points. - **Biggest hard flaw**: single case, author as developer, external validity limited. - **Must do before submission**: (a) external contrast project double coding (Cohen's κ [RESULT NEEDED]); (b) claim drift curve quantitative.

Appendix E Design Principles and Term Summary

E.1 Five Evidence-Grading Design Principles

From this repo's practice, five principles of universal meaning for AI co-development can be refined:

1. **Grades flow with data**: letting function return values carry evidence_grade fields, not handwriting grades afterward in documentation; grades are part of data, not part of narrative.
2. **Three-layer consistency**: code, reports, documentation point to the same grade; wanting to upgrade a claim must simultaneously change three places, and these have CI aligning.
3. **No benchmark no claim**: numbers without reports7 JSON and tests not written into conclusions; baseline regression gates prevent silent regression.
4. **Keep negative results**: these "ugly" numbers -0.508 , 0.5 , $1.59\times$ peak define system capability boundaries, deleting them more misleading than keeping.
5. **Conventions need test guarding**: the version-number incident proves, "we agree single source" does not equal "the system guarantees it"; conventions must be enforced by test assertions.

E.2 Three-Level Evidence Label Terms

- **verified**: fixed-seed, rerunnable, test-guarded measurement.
- **cpu-proto**: CPU deterministic prototype, Agents not real LLMs, no real network/GPU.
- **unverified**: external reporting/vendor caliber, only motivation, not cross-evidenced with this repo's numbers.

E.3 Relationship with Other Papers in This Matrix

This paper is the methodology-positioned one in the matrix. P1 (topology complexity) and P3 (fault governance RCT) provide "the object being measured," this paper provides the methodology of "how measurement is honestly executed." P1 insisting explicit/analytical grading, P3 insisting single-seed self-report and preregistration, both concrete practices of this paper's evidence discipline. The three do not repeat: P1 asks "structure cost," P3 asks "governance causality," P12 asks "how numbers stay honest."

E.4 One-Sentence Summary

In an AI-co-developed research engine, number trustworthiness is not writing-time decoration but a first-class output needing tests, gates, baseline regression, three-layer consistency to jointly guard; it can fail (e.g. the v-prefix incident), failure points must be honestly recorded, negative results must be kept—this is what this paper learned from a cross-30+-version real archive.

Appendix F Deep Analysis of the Version Incident and Failure Modes

F.1 Complete Causal Chain of the Version-Number Incident

For readers to fully understand this incident, we review its causal chain: (1) the release script uses a regex to replace version numbers in `udos7/__init__.py` and `pyproject.toml`; (2) this regex is written in the

"version number no v prefix" format; (3) but the actually written version number carried v prefix (e.g. "v7.4.3"); (4) the regex did not match, replacement silently failed; (5) because replacement failure throws no exception and there was no assertion checking "whether the replaced version number equals expected," 8 consecutive versions (v7.4.3 to v7.4.10)'s externally reported version number stayed at v7.4.2; (6) until v7.5.0 added a test asserting "the code-reported version number equals the expected version number," this silent error was exposed and fixed.

Each step of this causal chain deserves methodological reflection: silent failure (regex mismatch not erroring), missing assertions (no "whether after replacement equals expected" check), human expert bias (the regex writer assuming the version format as they thought). The three combined caused this 8-version metadata error.

F.2 Incident Lessons for Evidence Discipline

This incident has three lessons for evidence-grading methodology: first, **metadata is also data**. The version number is not an "irrelevant string," it is metadata for judging "which version this number comes from." If the version number is wrong for 8 versions, readers will mistake v7.5.0's numbers for v7.4.2's. Second, **conventions produce no guarantee**. "We agree version single source" is a promise, it does not error when the regex is wrong. Third, **tests are the only guard**. Only "test asserting version number equals expected" can catch this silent error. These three lessons are more persuasive than any positive case this paper tells—it comes from our own failure.

F.3 Prevention of Other Failure Modes

Besides the version-number incident, this repo identified other failure modes needing prevention: performance numbers silently regressing (prevented by baseline-JSON regression gates), prototype numbers upgraded to measured (prevented by three-layer consistency and evidence_grade fields), negative results deleted (prevented by "keep negative results" discipline), external numbers taken as this repo's numbers (prevented by unverified labels and "no cross-evidence" discipline), single seed spoken as population (prevented by "single-seed pilot" marking and "supplement ≥ 30 seeds before submission" discipline). These prevention mechanisms are not designed out of thin air but grew from errors that actually occurred or nearly occurred.

Appendix G Evidence-Grade Ledger of Kept Numbers

G.1 Key Numbers and Their Grades

For readers to see each number's evidence grade at a glance, this paper organizes key numbers as follows:

- 967,796 params, blind MSE 0.01037: v7_verification.json, **verified** (fixed seed, 21 gates all passed).
- acceleration skill -0.508, spring ω 0.426, other v2 0.82: v7_verification.json, **verified**.
- CPU-bound throughput 2 Agents 244 t/s ($1.59\times$), 8 Agents $0.695\times$: agent_scaling.json, **cpu-proto** (CPU deterministic simulation).
- I/O-bound $7.986\times$: agent_scaling.json, **cpu-proto/simulation** (asyncio.sleep simulating remote waiting).
- naive Sim2Real 0.5, robust 1.0, margin 0.22: splat_transfer_demo.json, **cpu-proto**.
- million-scale fan-in 9, rounds 14, star center 20,971,520: matrix_scale_demo.json, **cpu-proto** (analytical extrapolation).

- matrix 24/24 closing, 3 faults isolated, 5 reworks: matrix_scale_demo.json, **cpu-proto** (single-seed pilot).
- full regression 236 items all green: full_regression.log, **verified** (tests rerunnable).

This ledger itself is the evidence-grading methodology's product: every number attached its grade, readers not needing to guess.

G.2 Why Distinguish verified and cpu-proto

A common misunderstanding is: "as long as the number is in JSON and rerunnable, it is verified." This paper opposes this understanding. Rerunnable is only verified's necessary condition, not sufficient. A CPU deterministic simulation, even if rerunnable, does not represent real LLMs, real networks, real GPUs—so it is cpu-proto, not verified. Distinguishing the two is not to appear strict but to let readers know this number "what it can represent, cannot." Writing cpu-proto as verified is a number drift.

G.3 How External Numbers Are Handled

All external industry numbers (e.g. some Agent platform's scale, some model's capability claims), this paper only appears in introduction and discussion, marked unverified, reporting caliber, not independently verified, and not cross-evidenced with this repo's numbers. This is not distrust of externals but evidence discipline consistency: external numbers we cannot rerun, cannot verify, so they cannot be placed at the same evidence grade as this repo's rerunnable numbers.

Appendix H Methodology Limitations and Submission Positioning

H.1 Single-Case Limitation

This paper's most fundamental limitation is single case, author as developer. We cannot exclude "this project keeping numbers honest partly because it is small, research-positioned, author values number trustworthiness." In a project with greater commercial pressure, larger team, more collaborators, evidence discipline execution cost may be higher, bypass temptation greater. This paper does not claim this method works in all scenarios, only reporting its form in this one project.

H.2 No External Contrast

This paper did not contrast this project with other similar open-source Agent projects, so cannot answer "whether evidence grading really makes projects more honest than those without it." This contrast needs finding 3–5 similar projects, coding their documentation claims' evidence-grade completeness, double coding and reporting κ . Before then, this paper can only be a single-case empirical report, not making causal claims.

H.3 Submission Positioning

Based on the above limitations, this paper's submission positioning is empirical report tracks (e.g. ICSE SEIP, MSR, EMSE), not complete causal research. We expect reviewers to question "single case, self-report," hedging by publishing all archives, keeping negative results and incidents, clearly marking single seed and to-supplement. We do not predict acceptance.

H.4 Claims Not Made

To avoid overclaiming, this paper explicitly does not: claim "evidence grading works in all AI co-development projects"; claim "three-level labels are the only correct scheme"; claim "this paper's

method eliminates all number drift"; claim "the author himself never exaggerates in documentation." We only report: in this one project, this method looks like this, cost how much, failed where.

Appendix I Concrete Cases of Negative-Result Keeping

I.1 Why Negative Results Have Value

In research software, negative results are often deleted as "failure." But negative results' value lies in: they define system capability boundaries. A negative result "acceleration parameter not identifiable (skill -0.508)" tells us, under current observation conditions, not to estimate this parameter; a negative result "naive Sim2Real success 0.5" tells us to keep a safety margin when transferring; a negative result "CPU-bound 8 Agents反而 slower (0.695×)" tells us not to blindly add concurrency. If deleting these negative results, documentation becomes "prettier" but readers misled.

I.2 Isomorphism of Negative Results and "Honest Failure"

This paper emphasizes "keeping negative results" and P3's "honest failure (kill-switch)" are isomorphic: the former keeping at the documentation layer the number "this capability does not work," the latter keeping at the runtime layer the state "this task cannot be done." Both are "honesty over beauty." Evidence discipline is consistent at documentation and runtime layers.

I.3 Cost of Not Keeping Negative Results

If this repo deleted -0.508 and only reported 0.426 and 0.82, readers would think all parameters identifiable; deleted 0.5 and only reported 1.0, readers would think Sim2Real naturally feasible; deleted 0.695 and only reported 1.59, readers would think more Agents faster. Each of these misleads may make downstream engineering decisions wrong. Keeping negative results essentially blocks these misleads outside documentation.

Appendix J Unification of Evidence Grading and This Paper Matrix

J.1 Three Papers Share the Same Evidence Discipline

This matrix's P0 three papers (P1 topology complexity, P3 fault governance RCT, P12 evidence grading) share the same evidence discipline: all numbers attached verified/cpu-proto/unverified three-level labels; single-seed pilot marked, ≥ 30 seeds supplemented before submission; external numbers marked unverified and not cross-evidenced; not fabricating p values/CI/sample sizes/DOI/quartiles. P12 is this discipline's methodology meta-paper, P1 and P3 its two practice cases.

J.2 How Evidence Discipline Protects P1

In P1, evidence discipline manifests as explicit/analytical grading: million-scale fan-in 9, rounds 14 clearly marked analytical extrapolation, not "million Agents measured." This prevents closed-form extrapolation passing off as measured.

J.3 How Evidence Discipline Protects P3

In P3, evidence discipline manifests as single-seed pilot marking and preregistration: 24/24 closing clearly marked single-seed point estimate, full-factorial multi-seed to supplement. This prevents single-seed demos passing off as population causal conclusions.

J.4 Meaning of Unification

Putting the three papers together, evidence discipline is not some paper's decoration but the whole matrix's bottom method. It guarantees: whichever paper reports what number, readers immediately know this number's evidence grade, rerunnability and boundary. This is P12's value as methodology meta-paper.

Appendix K Practical Checklist for AI Co-developers

K.1 Starting Stage

If you are using AI to co-develop a research project, start with: first, add an `evidence_grade` field to each function return value; second, let report scripts write the whole dictionary (including grade) into JSON, not handwrite numbers; third, in documentation read numbers from JSON and preserve grade markings as-is. These three things cost little but immediately suppress "documentation writing prototype as measured."

K.2 Growth Stage

When the project iterates to dozens of versions, add: fourth, baseline-JSON regression gates—each release comparing key metrics, silent regression failing the gate; fifth, version-number assertion tests—asserting code-reported version number equals expected; sixth, keep negative results—writing "does not work" numbers into documentation rather than deleting. These three prevent silent regression, metadata errors and negative-result loss.

K.3 Mature Stage

When the project shares externally, add: seventh, resource gate ledger—writing clearly "what cannot be done" (e.g. needing GPU, API keys, real machines); eighth, external numbers all marked unverified and not cross-evidenced; ninth, single-seed pilots clearly marked, multi-seed supplemented before submission. These three prevent prototypes passing off as production, external number pollution, single seed passing off as population.

K.4 Why This Checklist

This checklist is not designed out of thin air, each item from this repo's errors that actually occurred or nearly occurred: version-number assertion tests from the v-prefix incident; baseline regression gates from worry about silent performance regression; keeping negative results from these real numbers -0.508 , 0.5 , $1.59\times$; resource gate ledger from GPU/API keys/real machines' real non-achievement. This is a checklist "grown from failure."

Appendix L Conclusion

L.1 Back to the Starting Point

This paper started from the problem of "number drift in AI co-development," reporting a cross-30+-version real engineering archive. We found the key to suppressing number drift is not smarter human authors but stronger institutions: letting evidence grades flow with data, consistent across code-reports-documentation, test guarded in CI; keeping negative results; honestly recording the method's own failure points.

L.2 Most Counter-Intuitive Finding

This paper's most counter-intuitive finding is "evidence discipline itself can fail"—the version-number incident proves even the most basic metadata like "version single source" needs test guarding, otherwise silently wrong for 8 versions. This reminds us: do not take any "convention" as guarantee. Conventions are only intent, tests are guards.

L.3 One Sentence to Readers

If you are using AI to co-develop a research project, remember: your numbers' trustworthiness is not writing-time decoration but a first-class output needing institutional guarding. Keep negative results, write out failure points, mark external numbers clearly—these "ugly" honesties win trust more than any beautiful performance claim.

Appendix M Glossary and Version Snapshot

M.1 Glossary

- **Number drift:** the phenomenon of prototype numbers quietly upgraded to "measured" in documentation.
- **Three-level evidence labels:** verified / cpu-proto / unverified.
- **Three-layer consistency:** code, reports, documentation pointing to the same evidence grade.
- **Baseline-JSON regression gate:** the CI gate comparing key metrics on release, preventing silent regression.
- **No benchmark no claim:** numbers without reports7 JSON and tests not written into conclusions.
- **Claim drift curve:** the curve obtained by cross-version coding of quantitative claim grade changes.

M.2 Version Snapshot

Version snapshot when this paper was written: UDOS Reasoning Engine v7.5.0 (main tag v7.5.0, commit 1a71270), full regression 236 items all green, CHANGELOG 38 version entries, Python 3.12.11, PyTorch 2.14.0+cpu, 2 threads, Linux CPU. All numbers from reports7/*.json, sha256 see Appendix A. This snapshot guarantees this paper's cited numbers independently recomputable at submission.

Appendix N Expected Dialogue with Related Reproducibility Literature

At submission, this paper expects dialogue with three types of literature: first, ML reproducibility crisis and model card/datasheet literature, this paper will point out it transformed model-card ideas from "single artifact" to "long-cycle multi-version, data-flowing three-level labels"; second, AI-assisted software engineering empirical studies, this paper will point out it focuses on the neglected dimension of "documentation numbers drifting over time"; third, empirical software engineering case study methods, this paper will adopt longitudinal single-case, public-archive, counter-example-keeping methods and self-report single-case and self-report bias. These three dialogue stances need refinement after literature search, currently only expected directions.

Appendix O Claims Not Made and Submission Positioning Reaffirmed

To avoid overclaiming, this paper explicitly does not make the following claims: does not claim three-level evidence labels are the only correct scheme; does not claim evidence grading works in all AI co-development projects; does not claim this paper has eliminated number drift; does not claim the author

himself never exaggerates in documentation. This paper only honestly reports: in this cross-30+-version research engine, this evidence-grading method looks like this, cost how much, failed where. Submission positioned as empirical report tracks, quartiles/impact factors/deadlines all to check, this paper does not predict acceptance.

Appendix P Author-as-Developer Bias Self-Report

Finally, this paper must self-report a fundamental bias: author as developer. This means we are more familiar and tolerant of this repo's numbers, possibly in coding tending to upgrade numbers "we think correct." This paper hedges in three ways: all coding pointing to concrete files and fields (Appendix G ledger); keeping negative results and incidents (not only telling success); for doubtful claims tending to downgrade rather than upgrade. But these hedges cannot completely eliminate bias. Readers should take this paper as an empirical report from a developer with bias self-report, not an audit from an independent third party. This self-report itself is part of evidence discipline—honestly reporting one's stance more credible than pretending neutrality.

Appendix Q Conclusion

Today as AI co-development increasingly popularizes, research software trustworthiness faces a new threat: numbers no longer only handwritten crooked but fluently, automatically "generated" to look reasonable. To cope with this threat, one cannot rely on more careful humans but stronger institutions. This paper, from a cross-30+-version real archive, refined an engineering method of three-level evidence grading, three-layer consistency, data flowing, test guarding, negative result keeping, honest failure recording. We do not claim it perfect, we only honestly report it looks like this, cost how much, broke where. This is the sample this paper wants to leave for this era. This paper believes honesty itself is an engineering output. In this era of AI auto-generating narrative, treating number trustworthiness as a first-class goal, more than treating it as writing decoration, lets research software go farther. This paper also welcomes the community to test this paper with the same public-archive standard: all numbers point to reports⁷ concrete fields, all negative results and incidents public, readers can recompute themselves. Before submission, this paper will complete external literature search formulas and candidate directions, and when conditions permit supplement multi-seed contrasts, to upgrade the current single-case self-report into a more contrastive empirical study. Before then, this paper keeps its positioning as a single-case empirical report, not making universal causal claims. End of paper. May this paper become an honest sample of reproducible engineering in the AI co-development era.